# Colonies de fourmis et problème du voyageur de commerce

Précision contre temps de calcul : quand utiliser l'ACO ?

Ivano Balboni  Quentin Wache

Novembre 2026

# Question et plan

Comment une colonie de fourmis trouve-t-elle de bonnes tournées, comment ses paramètres règlent le compromis exploration/exploitation, et quand vaut-elle la peine face aux méthodes classiques ?

**Partie 1 : comment la colonie apprend**

- Le modèle ACO, avec démonstration
- Exploration contre exploitation
- Influence des paramètres
- Face au hasard et à l'optimum exact
- Convergence selon la taille, complexité

**Partie 2 : quand utiliser l'ACO ?**

- Comparaison avec les méthodes classiques
- Précision contre temps de calcul
- TSP dynamique
- Cas d'usage

# Partie 1 : comment la colonie apprend

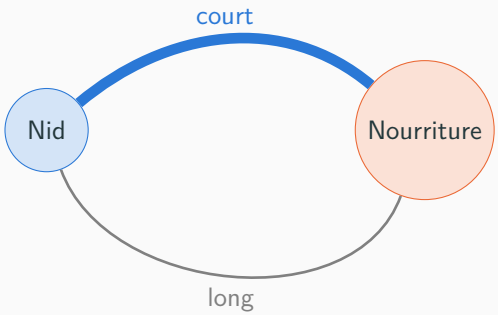
# Le voyageur de commerce (TSP)

- $n$ villes, distances $d_{ij}$
- Trouver le cycle le plus court qui passe une fois par chaque ville
- Problème NP-difficile : aucun algorithme polynomial connu
- Nombre de tournées distinctes : $\dfrac{(n-1)!}{2}$

| $n$ | tournées possibles |
|---|---|
| 10 | $1{,}8 \times 10^{5}$ |
| 20 | $6{,}1 \times 10^{16}$ |
| 30 | $4{,}4 \times 10^{30}$ |
| 50 | $3{,}0 \times 10^{62}$ |
| 100 | $4{,}7 \times 10^{155}$ |

À un milliard de tournées par seconde, $n = 20$ prend déjà 2 ans. Il faut donc choisir entre exactitude et temps de calcul.

# Inspiration : l'expérience du double pont




- Les fourmis déposent une phéromone sur leur chemin
- Sur le chemin court, les allers-retours sont plus rapides ⇒ plus de dépôt
- Plus de phéromone ⇒ plus de fourmis : rétroaction positive
- L'évaporation efface les mauvais chemins

Deneubourg et al. (1990) ; communication indirecte par l'environnement = *stigmergie*

# Ant System (Dorigo et al., 1996)

À chaque itération, $m$ fourmis construisent chacune une tournée.

**1. Choix de la ville suivante** (fourmi $k$ en $i$, villes non visitées $\mathcal{N}_k$) :

$$p_{ij} = \frac{\tau_{ij}^{\alpha}\ \eta_{ij}^{\beta}}{\sum_{l \in \mathcal{N}_k} \tau_{il}^{\alpha}\ \eta_{il}^{\beta}} \qquad \eta_{ij} = \frac{1}{d_{ij}}$$

**2. Mise à jour des phéromones** (après toutes les fourmis) :

$$\tau_{ij} \leftarrow \underbrace{(1-\rho)\,\tau_{ij}}_{\text{évaporation}} + \underbrace{\sum_{k} \frac{Q}{L_k} \mathbb{1}[(i,j) \in T_k]}_{\text{dépôt}}$$

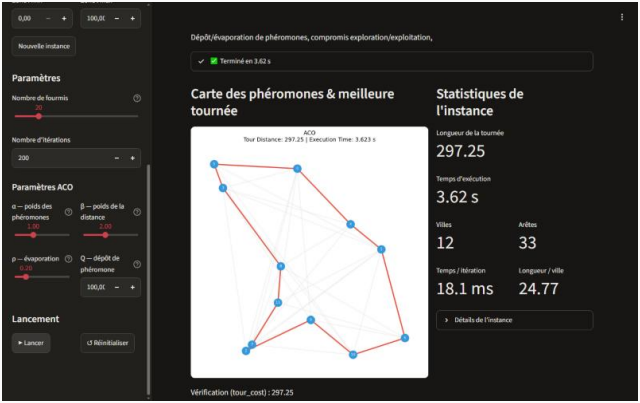
$\alpha$ : poids de la mémoire collective

$\beta$ : poids de la distance (glouton)

$\rho$ : vitesse d'oubli

$m$ : nombre de fourmis, $L_k$ : longueur de la tournée $k$

# Démonstration

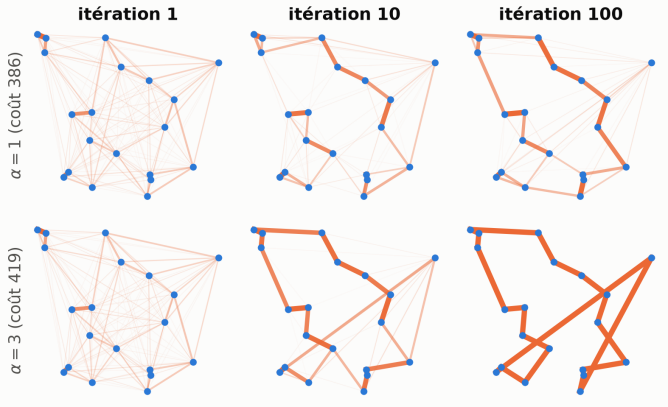



**En direct, 20 villes :**

1. Réglages de référence : $\alpha = 1$, $\beta = 2$, $\rho = 0{,}2$
2. $\alpha = 3$ : la colonie se fige sur une piste trop tôt
3. $\beta = 0$ : sans la distance, presque du hasard
4. Carte des phéromones : curseur d'itération

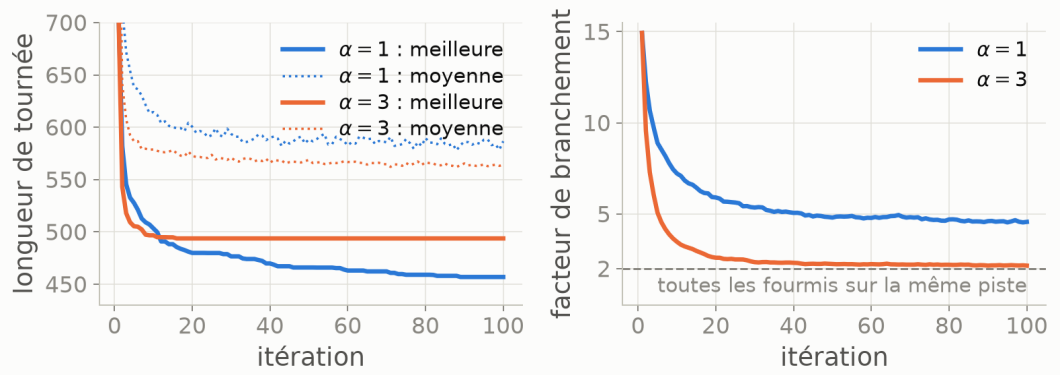
`using-aco-for-tsp-faqcu8kd7htkggervtpugk.streamlit.app`

# Exploration contre exploitation (1/2) : les pistes



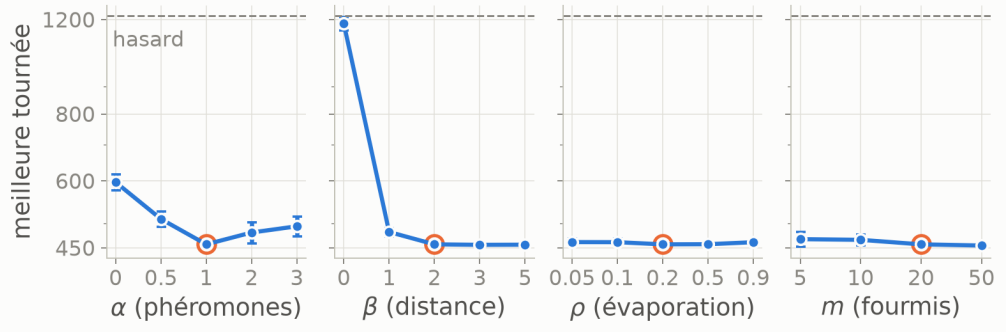

Épaisseur d'une arête = sa phéromone (normalisée à chaque itération), 20 villes. $\alpha = 1$ : plusieurs pistes coexistent et la tournée se dessine peu à peu (exploration). $\alpha = 3$ : une piste s'impose dès l'itération 10 et se fige sur une tournée qui se croise (convergence prématurée).

# Exploration contre exploitation (2/2) : la mesure




30 villes, 10 graines. $\alpha = 3$ converge en $\approx 10$ itérations mais s'arrête à 494 ; $\alpha = 1$ met $\approx 40$ itérations et atteint 457. Le facteur de branchement (nombre moyen d'arêtes encore attractives par ville) tombe vers 2 : la carte *montre* le blocage, ce facteur le *mesure*.

# Influence des paramètres (30 villes, 10 graines)

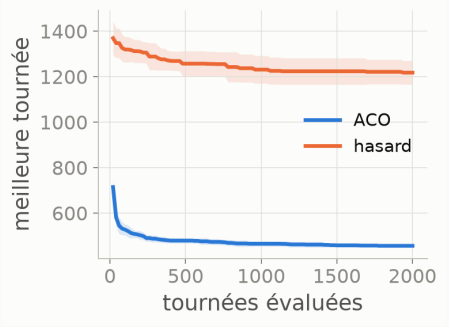



- $\beta = 0$ (distance ignorée) : presque aussi mauvais que le hasard
- $\alpha = 0$ (pas de mémoire) : 597 ; $\alpha$ trop grand : convergence prématurée
- $\rho$ : peu d'effet ici ; plus de fourmis : résultats plus réguliers (é.-t. 14,6 $\rightarrow$ 1,1)

Cercle orange : réglage de référence. Convergence prématurée ($\alpha = 3$) : voir la démo et l'annexe ; détail interactif : page « Analyse » de l'application.

# L'ACO face au hasard et à l'optimum exact

**Contre le hasard** (30 villes, même budget)




ACO : **457** ; hasard : 1218

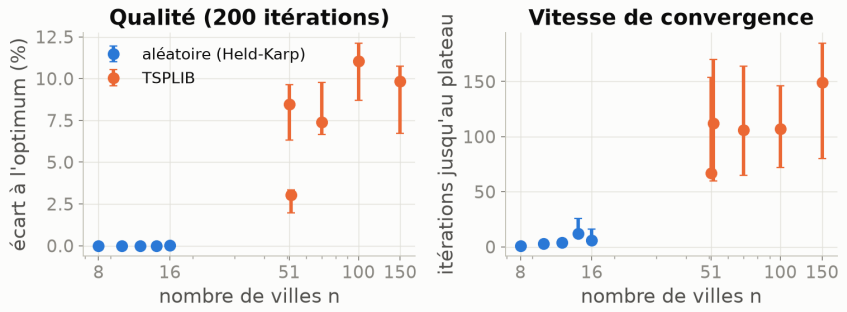
**Contre l'optimum exact** (Held-Karp)

| $n$ | Held-Karp | optimum atteint | itérations |
|---|---|---|---|
| 8 | 0,004 s | **5/5** | 2 |
| 10 | 0,02 s | **5/5** | 3 |
| 12 | 0,07 s | **5/5** | 5 |
| 14 | 0,3 s | **5/5** | 19 |
| 16 | 1,2 s | 0/5 (à 0,01 %) | – |

Ant System, 20 fourmis, 5 graines ; itérations : médiane avant l'optimum.

L'ACO trouve l'optimum en quelques itérations jusqu'à 14 villes ; à 16 villes, il s'arrête tout près sans l'atteindre : aucune garantie.

# Vitesse de convergence et qualité selon la taille




- Ant System, 20 fourmis, 200 itérations, 5 graines. Jusqu'à 16 villes : optimum (ou presque) en moins de 20 itérations
- De 51 à 150 villes : 3 à 11 % au-dessus de l'optimum ; la colonie atteint son plateau après 70 à 150 itérations, puis n'améliore plus
- Plus $n$ grandit, plus la convergence est lente et le résultat éloigné de l'optimum : l'Ant System seul ne suffit pas (partie 2 : ajouter une recherche locale)

# Complexité d'une itération d'ACO

- Construction : $m$ fourmis $\times$ $(n-1)$ pas $\times$ $O(n)$ candidats

$$\Rightarrow O(m\,n^2)$$

- Évaporation $O(n^2)$, dépôt $O(m\,n)$
- Total pour $T$ itérations : $\boxed{O(T\,m\,n^2)}$
- Mémoire : matrice des phéromones $O(n^2)$

**Prédictions testables**

- $m$ et $T$ fixés : temps $\propto n^2$ (pente 2 en log-log)
- 2-opt (listes de voisins) : $O(n)$ mouvements à $O(k)$ chacun $\Rightarrow$ la construction $O(m\,n^2)$ reste dominante
- Held-Karp : temps $\times \approx 2$ par ville ajoutée

# Dynamique des phéromones et convergence

**Évaporation seule :** $\tau_t = \tau_0(1-\rho)^t$

$\Rightarrow$ la colonie « oublie » en environ $1/\rho$ itérations $\quad (\rho = 0{,}2 : 5 \text{ it.} ; \rho = 0{,}02 : 50 \text{ it.})$

**Dépôt constant $\Delta$ :** $\tau_t \to \Delta/\rho$ (point fixe)

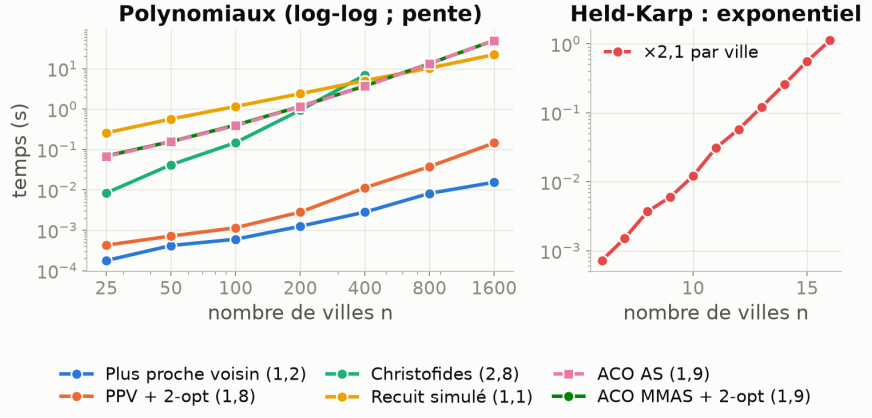
**Résultats de convergence**

- Gutjahr (2000) : un Ant System modifié trouve l'optimum avec une probabilité qui tend vers 1
- Stützle & Dorigo (2002) : avec $\tau_{\min} > 0$, la probabilité de trouver l'optimum tend vers 1

**Mais**

Garanties asymptotiques uniquement : aucun temps polynomial (le TSP est NP-difficile). La vraie question est donc expérimentale : quelle précision pour quel temps ?

# Temps de calcul en fonction de $n$




Pentes (entre parenthèses) proches de la théorie : 2 pour l'ACO et PPV + 2-opt, 3 pour Christofides.
Held-Karp : $\times 2{,}1$ par ville, 1,1 s à $n = 16$, $\approx$ 11 h et plus de 100 Go à $n = 30$.

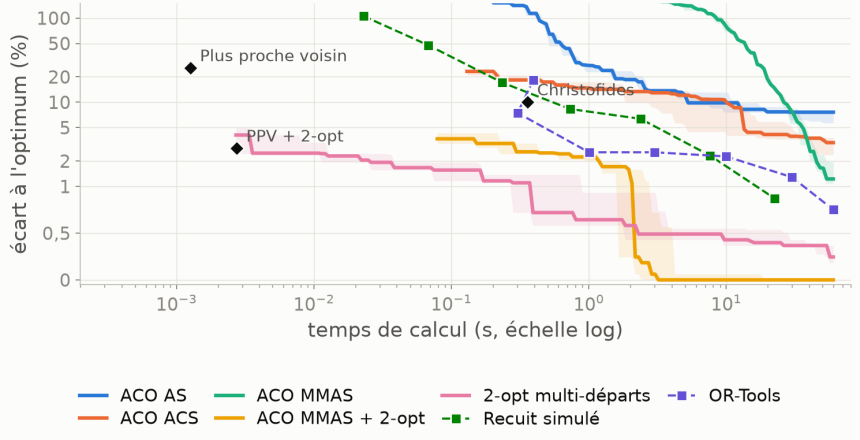
# Partie 2 : quand utiliser l'ACO ?

# Les algorithmes comparés

| Algorithme | Famille | Complexité théorique | Garantie |
|---|---|---|---|
| Held-Karp | exact (prog. dynamique) | $O(n^2 2^n)$ | optimum |
| Plus proche voisin (PPV) | glouton | $O(n^2)$ | aucune |
| PPV + 2-opt | recherche locale | $O(k)$ par mouvement* | optimum local |
| Christofides | approximation | $O(n^3)$ | $\leq 1{,}5\times$ optimum |
| Recuit simulé | métaheuristique | $O$(itérations) | asymptotique |
| OR-Tools (Google) | solveur industriel | limite de temps | aucune |
| ACO (AS, ACS, MMAS) | métaheuristique | $O(T\,m\,n^2)$ | asymptotique |
| ACO MMAS + 2-opt | hybride | $O(T\,m\,n^2)$ + 2-opt | asymptotique |

$T$ : itérations, $m$ : fourmis. * listes des $k = 20$ plus proches voisins, comme dans ACOTSP (code de référence de Dorigo & Stützle). Témoin ajouté : 2-opt multi-départs (2-opt relancé depuis des tournées aléatoires), pour isoler l'apport de l'apprentissage.

# ch150 (150 villes) : précision en fonction du temps accordé




5 instances TSPLIB de 51 à 783 villes, budget 60 s, 3 graines, mêmes conditions pour tous ; les autres instances sont en annexe.

# Quel algorithme gagne, selon le temps disponible ?

| Budget | eil51 | kroA100 | ch150 | pcb442 | rat783 |
|---|---|---|---|---|---|
| 10 ms | m.-d. (0,6) | m.-d. (0,7) | m.-d. (2,8) | PPV (22) | PPV (26) |
| 0,1 s | m.-d. (0,2) | m.-d. (0,2) | m.-d. (1,6) | m.-d. (4,2) | m.-d. (6,2) |
| 1 s | m.-d. (0) | m.-d. (0) | m.-d. (0,7) | m.-d. (3,3) | m.-d. (5,4) |
| 10 s | m.-d. (0) | MMAS$^{=}$ (0) | MMAS (0,02) | m.-d. (2,9) | m.-d. (5,0) |
| 60 s | MMAS$^{=}$ (0) | MMAS$^{=}$ (0) | MMAS (0) | m.-d. (2,7) | OR-Tools (4,1) |

Écart à l'optimum en %, moyenne de 3 graines. m.-d. : 2-opt multi-départs ; MMAS : MMAS + 2-opt ; $^{=}$ ex æquo avec m.-d. (et OR-Tools sur kroA100 à 60 s).

- Un 2-opt rapide relancé depuis des tournées aléatoires gagne presque partout
- MMAS + 2-opt ne gagne que lorsqu'il a le temps de faire des centaines d'itérations : 730 en 60 s sur ch150, mais 45 seulement sur rat783
- Les ACO sans recherche locale ne gagnent jamais

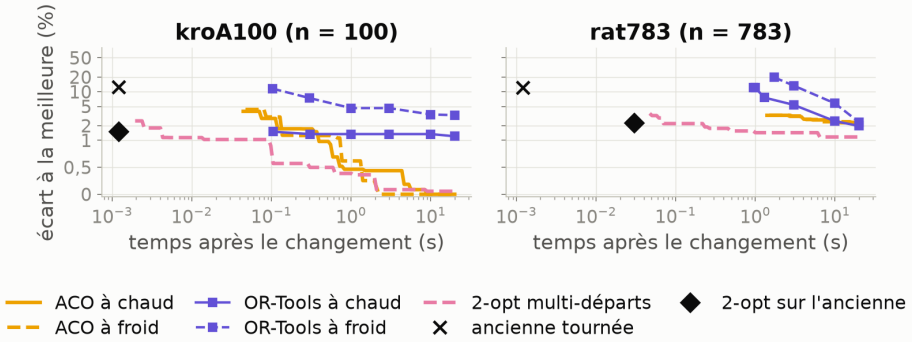
# Les phéromones aident-elles ? Le témoin 2-opt multi-départs

Même 2-opt, même temps, mais en partant de tournées aléatoires au lieu des tournées proposées par les fourmis.

| écart à l'optimum (%) | à 10 s | | à 60 s | |
|---|---|---|---|---|
| | 2-opt m.-d. | MMAS+2-opt | 2-opt m.-d. | MMAS+2-opt |
| eil51 | **0** | 0,08 | 0 | 0 |
| kroA100 | 0 | 0 | 0 | 0 |
| ch150 | 0,45 | **0,02** | 0,27 | **0** |
| pcb442 | **2,88** | 4,25 | **2,66** | 3,24 |
| rat783 | **4,99** | 6,34 | **4,66** | 5,25 |

- Les phéromones aident sur ch150 ; jusqu'à 100 villes, le 2-opt seul suffit
- Dès 442 villes, l'ACO fait trop peu d'itérations en 60 s (123 et 45) pour en profiter

# TSP dynamique : garder ses phéromones aide-t-il ?




- Après 20 s de calcul, 20 % des routes sont ralenties ($\times 1$ à $\times 3$) ; chaque méthode a 20 s pour réagir (médiane de 3 graines)
- Garder ses phéromones aide un peu (rat783 : 1,84 % contre 2,34 %), mais un 2-opt multi-départs relancé de zéro fait mieux (**1,17 %**)

# Quel algorithme pour quelle situation ? (TSP statique)

| Situation | Choix | Justification mesurée |
|---|---|---|
| $n \leq 15$, optimum exigé | Held-Karp | exact en $< 1$ s ; $\times 2$ par ville ensuite |
| Réponse en quelques ms | PPV + 2-opt | 3 à 7 % en 1 à 45 ms |
| Moins d'une seconde | 2-opt multi-départs | 0 à 5 % en 1 s, de 51 à 783 villes |
| Quelques secondes, $n \approx 150$ | MMAS + 2-opt | 0,02 % en 10 s sur ch150 (2-opt m.-d. : 0,45) |
| Grande instance, budget long | OR-Tools | meilleur sur rat783 en 60 s (4,1 %) |
| Garantie théorique exigée | Christofides | $\leq 1{,}5\times$ prouvé ; mesuré 8 à 14 % |
| ACO sans recherche locale | à éviter | toujours battu par une méthode avec 2-opt |

L'ACO joue le rôle de générateur de bons points de départ pour la recherche locale ; il ne paie que si le budget permet des centaines d'itérations.

# Où l'ACO a un avantage structurel

**Problèmes qui changent dans le temps**

- Routage réseau (AntNet, Di Caro & Dorigo, 1998) : les phéromones s'adaptent quand les coûts changent
- Notre mesure sur le TSP : gain faible

**Problèmes avec contraintes métier**

- Tournées de véhicules (capacités, créneaux), ordonnancement d'ateliers, affectation quadratique
- La contrainte s'ajoute dans la règle de choix de la fourmi

**Où ce n'est pas utile**

Plus court chemin sur un graphe fixe : Dijkstra est exact en $O((V+E)\log V)$. Une métaheuristique ne se justifie que si le problème est NP-difficile ou change dans le temps.

**Coût à prévoir**

Réglage des paramètres ($\alpha$, $\beta$, $\rho$) et des centaines d'itérations avant de battre un 2-opt multi-départs.

# Conclusion

**Comment la colonie apprend**

1. **L'ACO apprend :** 457 contre 1218 pour le hasard ; optimum exact atteint jusqu'à 14 villes, mais sans garantie (16 villes : 0,01 % au-dessus)
2. **Exploration contre exploitation :** $\alpha$ et $\beta$ comptent le plus ; $\alpha$ trop grand fige la colonie sur une piste (branchement $\rightarrow$ 2)
3. **Taille et complexité :** la convergence ralentit et la qualité baisse avec $n$ ; le coût mesuré suit la théorie $O(T\,m\,n^2)$

**Notre extension : quand utiliser l'ACO ?**

4. Sur le TSP classique, un 2-opt rapide relancé au hasard est souvent aussi bon ; MMAS + 2-opt ne gagne que s'il fait des centaines d'itérations (ch150 en 10 s)
5. L'ACO a sa place sur les problèmes à contraintes métier ; sur un TSP dynamique, garder ses phéromones aide peu

Merci ! Questions ? Démonstration : `using-aco-for-tsp-faqcu8kd7htkggervtpugk.streamlit.app`

# Références


- M. Dorigo, V. Maniezzo, A. Colorni. *Ant System : optimization by a colony of cooperating agents*. IEEE Trans. SMC-B, 26(1), 1996.
- M. Dorigo, L. M. Gambardella. *Ant Colony System*. IEEE Trans. Evol. Comp., 1(1), 1997.
- T. Stützle, H. Hoos. *MAX-MIN Ant System*. Future Gen. Comp. Systems, 16(8), 2000.
- W. J. Gutjahr. *A graph-based Ant System and its convergence*. Future Gen. Comp. Systems, 16(8), 2000.
- T. Stützle, M. Dorigo. *A short convergence proof for a class of ACO algorithms*. IEEE Trans. Evol. Comp., 6(4), 2002.
- J.-L. Deneubourg, S. Aron, S. Goss, J. M. Pasteels. *The self-organizing exploratory pattern of the Argentine ant*. J. Insect Behavior, 3, 1990.
- G. Di Caro, M. Dorigo. *AntNet : distributed stigmergetic control for communications networks*. JAIR, 9, 1998.
- N. Christofides. *Worst-case analysis of a new heuristic for the travelling salesman problem*. Rapport, Carnegie-Mellon University, 1976.
- M. Held, R. M. Karp. *A dynamic programming approach to sequencing problems*. J. SIAM, 10(1), 1962.
- M. Dorigo, T. Stützle. *Ant Colony Optimization*. MIT Press, 2004.
- G. Reinelt. *TSPLIB*. ORSA J. Computing, 3(4), 1991.

# Annexe : variantes testées

**AS** Ant System : toutes les fourmis déposent

**ACS** Ant Colony System (Dorigo & Gambardella, 1997) : avec probabilité $q_0 = 0{,}9$, choix de la meilleure arête ; mise à jour locale pendant la construction

**MMAS** MAX-MIN Ant System (Stützle & Hoos, 2000) : seule la meilleure fourmi dépose, $\tau \in [\tau_{\min}, \tau_{\max}]$ avec $\tau_{\max} = \frac{1}{\rho L^*}$, réinitialisation si la colonie stagne

**+ 2-opt** Recherche locale appliquée aux meilleures tournées de chaque itération

**Point commun**

Ce sont des algorithmes anytime : on peut les arrêter à tout moment, et la solution s'améliore avec le temps accordé.

# Annexe : protocole

**Expérience 1 : complexité**

- Instances aléatoires, $n = 25$ à $1\,600$
- Held-Karp : $n = 6$ à $16$
- ACO : 25 fourmis, 10 itérations
- Pente mesurée en log-log sur les grands $n$

**Expérience 2 : précision vs temps**

- TSPLIB à optimum prouvé, de 51 à 783 villes : eil51, kroA100, ch150, pcb442, rat783
- Budget de 60 s, 3 graines par algorithme aléatoire
- On enregistre la meilleure tournée au cours du temps

**Trois types d'algorithmes**

- *Un coup* (un point) : PPV, PPV + 2-opt, Christofides
- *Budget réglable* : recuit simulé (30 à 100 000 $n$ itérations), OR-Tools (0,1 à 60 s)
- *Anytime* (une courbe) : ACO, 2-opt multi-départs

**Équité**

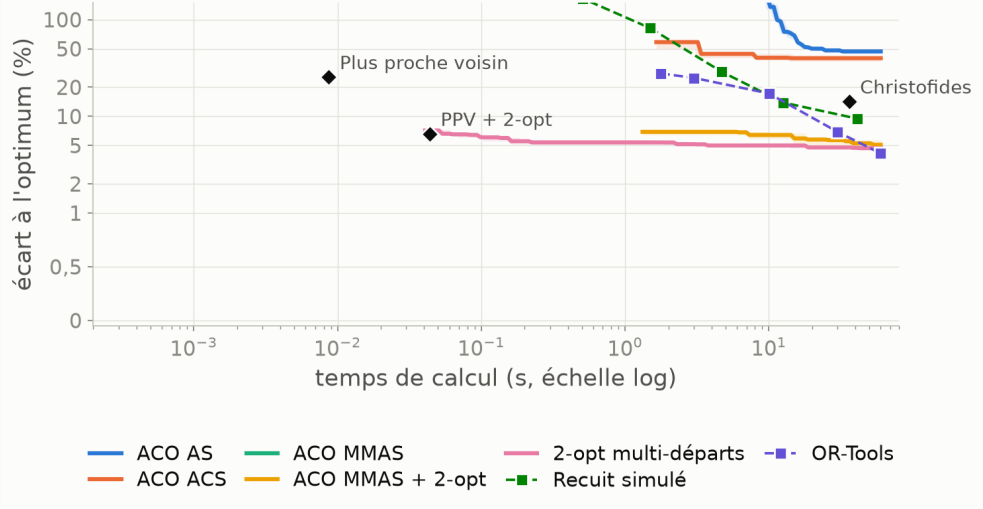
- Tous les algorithmes sur la même machine, dans les mêmes conditions
- Même budget de temps pour tous
- Paramètres standards de la littérature, sans réglage par instance

# Annexe : pentes mesurées vs théorie

| Algorithme | Théorie | Mesuré | Lecture |
|---|---|---|---|
| Christofides | 3 | **2,8** | conforme |
| ACO AS ($m$, $T$ fixés) | 2 | **1,9** | conforme |
| ACO MMAS + 2-opt | 2 | **1,9** | la construction domine |
| PPV + 2-opt | 2 | **1,8** | PPV $O(n^2)$, 2-opt $O(n)$ mouvements |
| Plus proche voisin | 2 | 1,2 | boucle interne vectorisée |
| Recuit simulé | 1 à 2 | 1,1 | coût par itération quasi constant |
| Held-Karp | $\times 2{,}3$ / ville | $\times$ **2,1** | exponentiel, conforme |

- Plus proche voisin : une opération sur $n$ valeurs coûte presque autant pour $n = 25$ que pour $n = 1\,600$ (coûts fixes) ; la théorie décrit le comportement pour $n$ grand.
- Held-Karp : 1,1 s à $n = 16$ ; extrapolé à $n = 30$ : $\approx 11$ h et plus de 100 Go de mémoire.

# Annexe : rat783 (783 villes)



# Annexe : écart à l'optimum (%) avec un budget de 1 s

| | eil51 | kroA100 | ch150 | pcb442 | rat783 |
|---|---|---|---|---|---|
| Plus proche voisin | 19,9 | 30,7 | 25,5 | 22,1 | 25,5 |
| PPV + 2-opt | 3,8 | 4,0 | 2,8 | 7,0 | 6,6 |
| Christofides | 8,4 | 9,4 | 10,0 | – | – |
| Recuit simulé | 2,5 | 3,2 | 8,2 | 49,2 | 163 |
| OR-Tools | 1,9 | 0,8 | 2,5 | 13,3 | – |
| 2-opt multi-départs | **0,0** | **0,0** | **0,7** | **3,3** | **5,4** |
| ACO AS | 9,4 | 17,5 | 25,9 | 345 | – |
| ACO ACS | 3,4 | 14,9 | 15,2 | 48,8 | – |
| ACO MMAS | 57,6 | 122 | 173 | 344 | – |
| ACO MMAS + 2-opt | 0,2 | 0,1 | 2,2 | 6,1 | – |

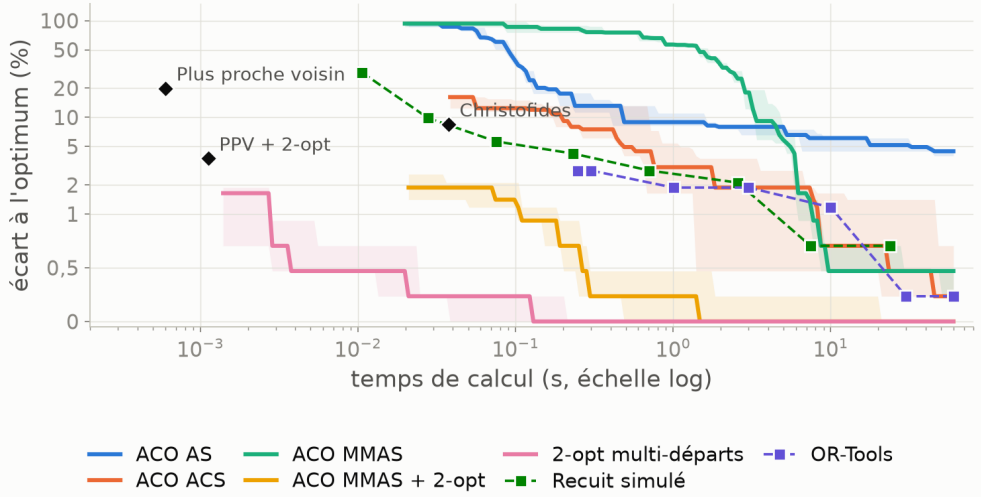
Moyenne sur 3 graines ; un algorithme compte si sa solution est obtenue avant le budget (+10 %) ; – : pas encore de solution.

# Annexe : écart à l'optimum (%) avec un budget de 60 s

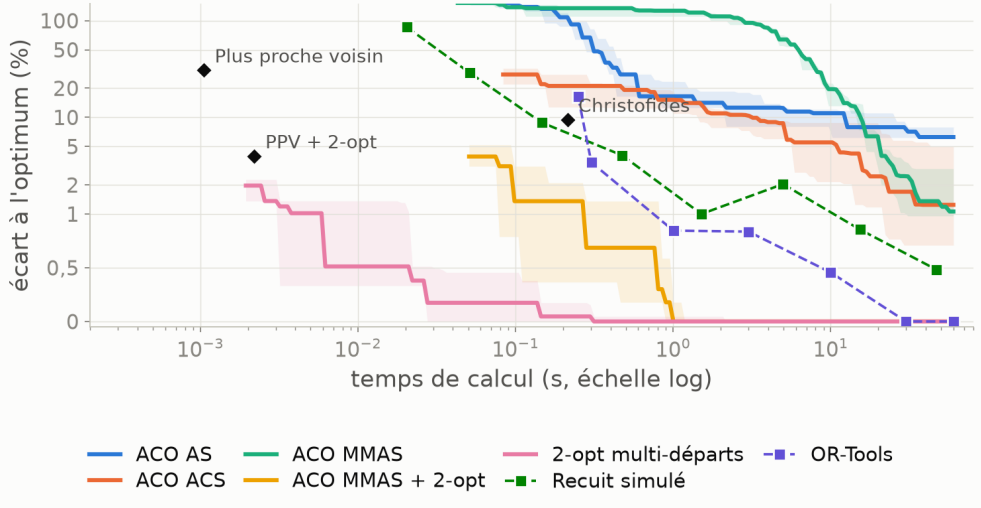
| | eil51 | kroA100 | ch150 | pcb442 | rat783 |
|---|---|---|---|---|---|
| Plus proche voisin | 19,9 | 30,7 | 25,5 | 22,1 | 25,5 |
| PPV + 2-opt | 3,8 | 4,0 | 2,8 | 7,0 | 6,6 |
| Christofides | 8,4 | 9,4 | 10,0 | 8,0 | 14,3 |
| Recuit simulé | 0,9 | 0,7 | 0,9 | 3,2 | 9,0 |
| OR-Tools | 0,2 | **0,0** | 0,8 | 3,8 | **4,1** |
| 2-opt multi-départs | **0,0** | **0,0** | 0,3 | **2,7** | 4,7 |
| ACO AS | 4,4 | 6,8 | 7,0 | 38,8 | 46,5 |
| ACO ACS | 0,4 | 2,3 | 3,2 | 26,9 | 39,7 |
| ACO MMAS | 0,4 | 1,7 | 1,4 | 180 | 374 |
| ACO MMAS + 2-opt | **0,0** | **0,0** | **0,0** | 3,2 | 5,2 |

Itérations de MMAS + 2-opt en 60 s : 3 450 (eil51), 1 400 (kroA100), 730 (ch150), 123 (pcb442), 45 (rat783).

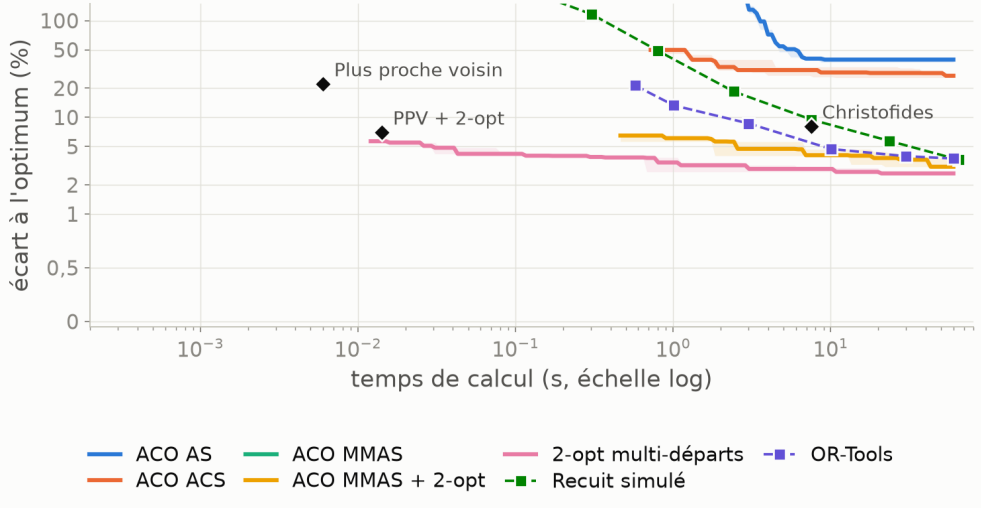
# Annexe : eil51 (51 villes)



# Annexe : kroA100 (100 villes)



# Annexe : pcb442 (442 villes)



# Annexe : reproduire les résultats

Code : `github.com/BalboniIvanoWacheQuentin/using-ACO-for-TSP`

- `python accuracy_time.py` : complexité et précision vs temps (`results/complexity.csv`, `anytime_<instance>.csv`) ; `python accuracy_time.py anytime rat783` pour une seule instance
- `python plot_accuracy_time.py` : figures et tableau par budget
- `python experiments.py` : étude des paramètres de l'ACO
- `python subject.py` : ACO face à Held-Karp et selon la taille
- `python dynamic.py` : TSP dynamique, reprise à chaud ou à froid (`results/dynamic_<instance>.csv`)
- `python slides/make_figures.py` : figures complémentaires
- `streamlit run streamlit.py` : démonstration interactive (aussi en ligne : `using-aco-for-tsp-faqcu8kd7htkggervtpugk.streamlit.app`)